



RAG Pipeline Evaluation Module - Redesigned System

Based on your request, I've reviewed and recreated the design for the "Design evaluation for RAG" project, focusing on an advanced evaluation module. This module helps users optimize RAG ingestion and retrieval pipelines by providing detailed, data-driven recommendations for parameters across 8 key components. The design emphasizes **static evaluation** (rule-based, heuristic-driven suggestions using benchmarks and best practices) and **dynamic evaluation** (iterative, experimental testing with user-provided sample data to measure real performance using metrics like ROUGE, BLEU, faithfulness, relevance, latency, and cost).

The system uses **multiple iterations** (typically 3-5 cycles) for dynamic evaluations, where parameters are adjusted progressively based on performance feedback (e.g., starting with baseline values, measuring metrics, and optimizing via grid search or Bayesian optimization). All evaluations are exposed via **API endpoints** for modularity, with an **overall recommendation API** that orchestrates them internally.

I've incorporated diagrams for visualization:

- **Architecture Diagram:** High-level system layers.
- **Flow Diagram:** End-to-end process flow.
- **API System Design Diagram:** Endpoint interactions.
- **Low-Level Design Diagram:** Evaluation logic flows with iterations.

(Architecture Diagram: Multi-layered system showing user input, evaluation engines, knowledge base, and output integration.)

Detailed User Input Requirements

To perform accurate evaluations, the system requires comprehensive user inputs. These are collected via a dedicated `POST /user-requirements` API (detailed later). The inputs influence all recommendations by weighting factors like accuracy, latency, cost, and data characteristics. Required fields include:

- **Use Case Details:** Description of the RAG application (e.g., document QA, knowledge base, customer support, code search, summarization).
- **Source Data Type:** Formats like PDF, Word, TXT, RDBMS tables (e.g., SQL), NoSQL (e.g., JSON documents), CSV, images, or web pages.
- **Sample Data:** Upload or provide 5-10 representative samples (e.g., text snippets, files, or database excerpts) for dynamic testing.

- **Data Volume:** Current total size (e.g., 1GB), expected growth (e.g., 10% monthly), and query volume (e.g., 100/day).
- **Priorities:** Ranked preferences (e.g., 1. Accuracy, 2. Latency, 3. Cost) on a 1-10 scale.
- **Cost Constraints:** Budget per query or monthly (e.g., \$0.01/query, optional).
- **Data Privacy:** Requirements like on-premise only, encryption needs, or compliance (e.g., GDPR).
- **Query Complexity:** Simple (keyword-based) vs. complex (multi-hop reasoning, multilingual).
- **Latency Tolerance:** Maximum allowed time (e.g., <200ms).
- **Accuracy Tolerance:** Minimum acceptable score (e.g., 85% on ROUGE-1).
- **Truthfulness Tolerance:** Minimum faithfulness score (e.g., 90%, measured via LLM-based evaluation).
- **Domain Specificity:** Field (e.g., medical, legal) and languages (e.g., English, multilingual).
- **Hardware Constraints:** Available resources (e.g., GPU availability, RAM).
- **Evaluation Dataset:** Optional ground-truth Q&A pairs for metrics like ROUGE/BLEU.
- **Custom Metrics:** User-defined weights for metrics (e.g., prioritize BLEU over ROUGE for summarization).

These inputs generate a unique `requirements_id` for session tracking.

(Flow Diagram: Step-by-step from user input to iterative evaluations and final recommendations, showing parallel API calls and iteration loops.)

The 8 Evaluation Components

Each component has:

- **Static Evaluation:** Heuristic-based suggestions from benchmarks (e.g., MTEB leaderboard for embeddings).
- **Dynamic Evaluation:** Iterative testing on sample data (3-5 iterations: baseline test → metric calculation → parameter adjustment → retest).
- **Iteration Logic:** Use grid search or optimization (e.g., Bayesian) to test parameter combinations. For example, start with defaults, measure metrics (ROUGE for relevance, BLEU for generation quality, latency timers), and iterate until convergence (e.g., <5% improvement or max iterations).
- **API Endpoint:** Input/output schemas.

1. Embedding Model Evaluation

Static: Recommend based on priorities (e.g., high-accuracy: text-embedding-3-large via MTEB scores).

Dynamic: Embed sample data with 3-5 models, measure semantic similarity (cosine), retrieval accuracy (precision@k), and iterate on parameters like batch size.

Iteration Logic: Test 3 batches (e.g., sizes 16/32/64), compute ROUGE on retrieved chunks vs.

ground truth, optimize for user priorities (e.g., if latency > tolerance, reduce batch size).

Parameters: Dimensions, batch size (16-64), normalization (true/false), GPU (true/false), half precision, num_workers (2-8).

API: POST /evaluate-embedding

Input: {requirements_id, sample_data, priorities}

Output: {recommended_model: "text-embedding-3-large", parameters: {batch_size: 32, ...}, metrics: {rouge: 0.85, latency: 150ms}, iterations: 4}

2. Chunking Strategy Evaluation

Static: Suggest based on data type (e.g., semantic chunking for narrative text via fixed heuristics).

Dynamic: Split sample data with varying parameters, embed/retrieve, measure BLEU/ROUGE on reconstructed text vs. original, iterate to minimize information loss.

Iteration Logic: 4 iterations testing chunk sizes (512-1500), overlaps (50-200); compute metrics, adjust (e.g., increase overlap if ROUGE < tolerance).

Parameters: Chunk size (512-2000), overlap (0-300), merge small chunks (true/false), min size (100), delimiter (e.g., "\n\n"), normalization (true/false), chunking method (fixed/semantic/recursive).

API: POST /evaluate-chunking

Input: {requirements_id, sample_data, data_type}

Output: {recommended_strategy: "semantic", parameters: {size: 1000, overlap: 200, ...}, metrics: {bleu: 0.92, info_loss: 5%}, iterations: 4}

3. Dimensions Optimization Evaluation

Static: Recommend reduced dimensions (e.g., 768 for semantic search) based on use case benchmarks, targeting <2% accuracy loss.

Dynamic: Apply PCA/truncation to embeddings of sample data, measure retrieval metrics (NDCG), iterate on reduction ratios.

Iteration Logic: 3 iterations starting at full dimensions, reduce by 25% each time, validate with ROUGE on queries; stop if accuracy drops below tolerance.

Parameters: Target dimensions (256-3072), reduction method (PCA/truncation), variance retained (0.95+).

API: POST /evaluate-dimensions

Input: {requirements_id, embedding_model, sample_data}

Output: {recommended_dimensions: 768, reduction_impact: {accuracy_loss: 1.5%, storage_savings: 75%}, metrics: {ndcg: 0.88}, iterations: 3}

4. LLM Model Evaluation for RAG Inference

Static: Suggest models like GPT-4o for accuracy or Llama-3 for cost based on use case (e.g., via GLUE benchmarks).

Dynamic: Generate responses on sample queries, score with ROUGE/BLEU/faithfulness (LLM-as-judge), iterate on models/parameters.

Iteration Logic: Test 3-5 models, measure metrics; if truthfulness < tolerance, switch to finer-tuned model and retest.

Parameters: Model name, temperature (0.1-1.0), top_p (0.8-1.0), max_tokens (512-4096).

API: POST /evaluate-llm

Input: {requirements_id, sample_queries, priorities}

Output: {recommended_model: "gpt-4o", parameters: {temperature: 0.7, ...}, metrics: {rouge: 0.87, faithfulness: 0.95}, iterations: 5}

5. Indexing Evaluation on Vector Databases

Static: Recommend HNSW for speed (e.g., in Qdrant) based on volume benchmarks.

Dynamic: Build indexes on sample embeddings, query with variations, measure recall/latency, iterate on parameters.

Iteration Logic: 4 iterations tuning ef_search (64-256); compute precision@k, adjust if recall < tolerance.

Parameters: Index type (HNSW/IVF/Flat), ef_search (64-512), ef_construction (100-500), M (8-64) for HNSW.

API: POST /evaluate-indexing

Input: {requirements_id, vector_db, sample_embeddings}

Output: {recommended_index: "HNSW", parameters: {m: 16, ef_search: 128}, metrics: {recall: 0.92, latency: 50ms}, iterations: 4}

6. Vector Database Evaluation

Static: Suggest Qdrant for medium scale or FAISS for local based on volume/privacy.

Dynamic: Load sample data into 3-4 DBs, run queries, measure throughput/cost, iterate on configs.

Iteration Logic: Test scalability by adding data batches, measure metrics; switch DB if latency exceeds tolerance.

Parameters: DB name (MongoDB, Qdrant, Milvus, Redis, FAISS), distance metric (cosine/euclidean), quantization (int8).

API: POST /evaluate-vectordb

Input: {requirements_id, data_volume, privacy_constraints}

Output: {recommended_db: "Qdrant", config: {distance: "cosine"}, metrics: {throughput: 1000qps, cost: \$0.05/query}, iterations: 3}

7. Retrieval Mechanism Evaluation

Static: Suggest hybrid search for complex queries based on DB (e.g., MMR in Qdrant).

Dynamic: Run retrieval on sample queries, score with MRR/NDCG, iterate on top_k/thresholds.

Iteration Logic: 5 iterations adjusting top_k (3-20); recompute ROUGE on retrieved contexts, optimize for accuracy/latency balance.

Parameters: Mechanism (similarity/hybrid/MMR), top_k (5-20), similarity_threshold (0.7-0.9), reranking (true/false).

API: POST /evaluate-retrieval

Input: {requirements_id, vector_db, sample_queries}

Output: {recommended_mechanism: "hybrid", parameters: {top_k: 10}, metrics: {mrr: 0.85, ndcg: 0.90}, iterations: 5}

8. Prompt Template Evaluation

Static: Suggest templates like few-shot for QA based on use case.

Dynamic: Generate responses with variations, score BLEU/ROUGE/faithfulness, iterate on templates.

Iteration Logic: Test 3-4 templates, measure metrics; refine placeholders if truthfulness < tolerance.

Parameters: Template string, shots (0-5), system instructions.

API: POST /evaluate-prompt

Input: {requirements_id, use_case, sample_queries}

Output: {recommended_template: "You are a helpful assistant. {context} Q: {query}",
metrics: {bleu: 0.88, faithfulness: 0.96}, iterations: 3}

(API System Design Diagram: Endpoints and internal calls from overall API to the 8 evaluation APIs.)

(Low-Level Design Diagram: Detailed logic flows for a sample evaluation (e.g., embedding) with iteration loops, metric calculations, and decision points.)

Overall Recommendation API

Endpoint: POST /recommend-pipeline

Purpose: Orchestrates the 8 APIs internally, runs iterations, and outputs a complete RAG/ingestion pipeline.

Input: {requirements_id} (triggers sequential/parallel calls to the 8 APIs).

Output: Integrated JSON with all recommendations, full config, estimated metrics (e.g., end-to-end ROUGE: 0.90, latency: 180ms), and implementation steps.

Internal Flow: 1. Validate inputs. 2. Parallel calls to 8 APIs. 3. Aggregate results with cross-validation (e.g., ensure embedding dimensions match vector DB). 4. Iterate 2-3 times on the full pipeline if overall metrics < tolerances.

Evaluation Documentation & Reasoning

Metrics Used: ROUGE (relevance), BLEU (generation quality), faithfulness/relevance (LLM-judged), precision/recall/MRR/NDCG (retrieval), latency/cost (measured).

Static Reasoning: Relies on benchmarks (e.g., MTEB for embeddings) and rules (e.g., if volume >10M, prefer Milvus).

Dynamic Reasoning: Real testing on samples; iterations use optimization to maximize a weighted score (e.g., $0.6accuracy + 0.3speed + 0.1*cost$).

Detailed Calculations: For example, in chunking: Compute ROUGE-1 = (overlapping n-grams / total n-grams); if <0.85, iterate by increasing overlap by 50. All iterations log metrics for transparency.

This redesigned system ensures optimal, user-tailored RAG pipelines through rigorous, iterative evaluations.